

CredX7i - Plano Técnico

Módulo de crédito multitenant Versão 1.0 - 27/08/2026 Documento de planejamento técnico. Não contém implementação.

1. Resumo executivo

O CredX7i é um módulo de crédito servido como aplicação web única para múltiplos clientes (tenants), com dados relacionais e upload de arquivos por cliente.

Decisões centrais deste plano:

#	Decisão	Justificativa resumida
1	Reaproveitar a arquitetura do WebCRM (Vercel + Supabase Postgres + Node/Express)	Já validada em produção, custo baixo, conhecimento acumulado
2	Tenancy em banco compartilhado com RLS , não multi-database	10 clientes e ~30 GB/ano não justificam multi-database; RLS dá isolamento no banco, não na aplicação
3	Arquitetura pool + silo : preparada desde já para mover um cliente específico para banco dedicado	Atende a exigência contratual futura sem refatoração
4	Isolamento imposto pelo Postgres (RLS) , substituindo a checagem por aplicação usada no WebCRM	Corrige a classe de falha já observada no WebCRM (roteador genérico expondo tabela fora do mapa de permissão)
5	Arquivos em Cloudflare R2 , não em Supabase Storage	Egress zero, custo ~15x menor no volume projetado, upload direto do browser
6	Autenticação própria mantida (padrão WebCRM), integrada ao RLS via <code>SET LOCAL</code>	Evita reescrever auth e evita custo adicional

Custo estimado em produção com os 10 clientes: ~US\$ 46/mês (detalhe na seção 12).

2. Requisitos e premissas

2.1 Confirmados pelo usuário

- **10 clientes** previstos no primeiro ano.
- **2 a 3 GB por cliente por ano**, predominantemente arquivos. Total ~30 GB/ano, ~90 GB acumulados no ano 3.
- **Cada usuário pertence a um único cliente.** Não há usuário compartilhado entre tenants.
- **Domínio de crédito**, com dados financeiros de pessoas. Sujeito a LGPD.
- **Nenhum cliente exige segregação hoje**, mas algum poderá exigir.

2.2 Premissas derivadas (a validar)

- O volume relacional (propostas, parcelas, pagamentos, histórico) fica abaixo de 2 GB no total por vários anos. Os 8 GB do plano Supabase Pro são folgados.
- O uso é comercial, logo Vercel Hobby não é elegível em produção.
- Não há requisito de residência de dados fora do padrão dos provedores.

2.3 Não-objetivos desta versão

- Não define telas, regras de negócio de crédito, motor de decisão ou integrações com bureaus. Este documento cobre plataforma e isolamento.
- Não define pipeline de CI/CD além do deploy automático nativo da Vercel.

3. Arquitetura de referência: o que vem do WebCRM

O WebCRM opera hoje com:

```
Navegador
  | HTTPS
  v
Frontend estático (React + Vite) --> projeto Vercel #1
  | fetch / API REST
  v
Backend Node.js + Express (TypeScript) --> projeto Vercel #2 (serverless)
  | driver pg, via connection pooler
  v
Postgres (Supabase)
+
Supabase Storage (anexos)
```

Características herdadas que **permanecem**:

- Dois projetos Vercel separados (frontend estático e backend serverless). O modelo de monorepo multi-serviço já foi tentado e abandonado no WebCRM.
- Driver `pg` com `types.setTypeParser(1700, parseFloat)` para colunas `NUMERIC`. Sem isso, valores monetários chegam como string e toda formatação vira no-op. Este é um erro já pago no WebCRM e deve nascer corrigido no CredX7i.
- Connection pooler do Supabase (Supavisor) em modo transaction, obrigatório para funções serverless.
- Autenticação própria: e-mail e senha, tabela de sessões, rate limiting nas rotas de login, revogação de sessões na troca de senha.
- Schema, views e triggers versionados em arquivos `.sql` no repositório.

3.1 O que muda, e por quê

Mudança 1 - Isolamento passa a ser responsabilidade do banco, não da aplicação.

No WebCRM, a permissão é checada na camada Express (`enforceMenuPermission` , mapa `MENU_BY_RESOURCE`), sobre um roteador REST genérico que expõe tabelas e views. A auditoria de segurança #2 encontrou um achado crítico exatamente nessa costura: a tabela `usuario_sesoes` não estava no mapa e ficou legível por qualquer usuário autenticado, expondo o token de sessão de todos os outros. Recursos fora do mapa continuam sem checagem.

Em aplicação de cliente único, esse tipo de falha é grave. Em multitenant, é vazamento entre clientes. A correção estrutural não é "lembrar de mapear toda tabela", é mover a fronteira para dentro do Postgres com Row Level Security. Com RLS ativo, uma tabela nova esquecida no mapa **não vaza**: a política nega por padrão.

Mudança 2 - Storage sai do Supabase e vai para o Cloudflare R2.

Detalhado na seção 9.

Mudança 3 - Toda tabela de negócio ganha `tenant_id`.

Inclusive nos futuros silos, onde tecnicamente seria redundante. Isso é o que torna a promoção de pool para silo uma operação de dump e restore, sem alteração de código.

Mudança 4 - O backend nunca conecta como dono do schema.

Conexão de aplicação usa um role restrito, sem `BYPASSRLS` e sem `SUPERUSER`.

4. Modelo de tenancy: pool + silo

4.1 Por que não multi-database agora

O usuário levantou multi-database por segurança e isolamento. É uma preocupação correta, mas o custo é desproporcional ao ganho neste porte:

- No Supabase, "um banco por cliente" na prática é "um projeto por cliente". O free tier permite 2 projetos e pausa por inatividade. Não é viável para produção.
- Toda migration precisa rodar N vezes, com N versões possíveis de schema em campo. Divergência de schema entre tenants é a principal fonte de incidente em produtos multi-database.
- Multiplica connection strings, secrets, backups, monitoração e provisionamento de cliente novo. Vira trabalho de plataforma, que hoje não existe.
- O ganho de segurança sobre RLS bem feito é menor do que parece: RLS é barreira no motor do banco, não condicional em código de aplicação.

4.2 O modelo adotado

Pool (padrão). Todos os tenants no mesmo Postgres, isolados por RLS.

Silo (sob demanda). Um tenant específico ganha banco dedicado quando houver exigência contratual ou regulatória. Mesmo schema, mesmo código.

Tenant registry. Uma tabela de controle mapeia cada tenant para o seu destino:

```

tenants
  tenant_id      uuid      PK
  slug           text      UNIQUE  -- usado no subdomínio
  nome           text
  modo           text      -- 'pool' | 'silo'
  db_secret_ref  text      NULL    -- nome da env var com a connection string do silo
  storage_prefix text      -- prefixo no bucket R2
  status         text      -- 'ativo' | 'suspense' | 'encerrado'
  criado_em      timestampz

```

A camada de acesso a dados resolve a conexão por request: `modo = 'pool'` usa o pool compartilhado; `modo = 'silo'` usa a connection string apontada por `db_secret_ref`. O restante do código não sabe a diferença.

Regra importante: `db_secret_ref` guarda o **nome** da variável de ambiente, nunca a connection string. Credencial de banco não fica em tabela.

4.3 Promoção de pool para silo

Procedimento previsto (a ser roteirizado na fase 5):

1. Provisionar novo Postgres e aplicar o schema versionado.
2. `pg_dump` filtrado por `tenant_id` do tenant alvo, ou `COPY ... WHERE tenant_id = ...` tabela a tabela em ordem de dependência.
3. Restaurar no banco novo.
4. Copiar o prefixo `tenant_id/` do bucket R2 para o bucket dedicado, se aplicável.
5. Janela de manutenção: marcar o tenant como `suspense`, sincronizar o delta, trocar `modo` para `silo`, reativar.
6. Após validação, expurgar as linhas do tenant no banco pool.

Como `tenant_id` existe em toda tabela, os passos 2 e 6 são mecânicos.

5. Stack e serviços

Camada	Escolha	Alternativa considerada	Motivo
Frontend	React + Vite, estático na Vercel	Next.js	Continuidade com o WebCRM; SSR não é necessário para app autenticado
Backend	Node.js + Express + TypeScript, serverless na Vercel	Next.js API routes	Continuidade; separação clara frontend/backend já validada
Banco	Postgres no Supabase (Pro)	Neon	Supabase já em uso, com Storage e painel SQL conhecidos
Pooling	Supavisor, modo transaction	Conexão direta	Obrigatório em serverless
Arquivos	Cloudflare R2	Supabase Storage	Egress zero e custo muito menor no volume projetado
Auth	Própria (padrão WebCRM) + RLS via <code>SET LOCAL</code>	Supabase Auth + claims no JWT	Evita reescrita e custo; ver 8.4
Segredos	Env vars da Vercel	Vault dedicado	Suficiente no porte atual

5.1 Roteamento de tenant

Subdomínio por cliente: `cliente.credx7i.com.br`, via wildcard domain na Vercel. Mais limpo para cliente externo do que `/t/cliente`, e permite personalização de marca por tenant no futuro.

O backend resolve o `slug` a partir do header `Host`, consulta o registry e injeta o `tenant_id` no contexto da requisição. **O `tenant_id` nunca vem de parâmetro enviado pelo cliente.** Ele vem da sessão autenticada, e o subdomínio serve apenas para selecionar a tela de login correta e validar coerência.

6. Modelo de dados

6.1 Convenções

- Toda tabela de negócio tem `tenant_id uuid NOT NULL REFERENCES tenants(tenant_id)`.
- Toda tabela de negócio tem RLS habilitado e forçado.
- Chaves primárias em `uuid` (evita colisão de ID na fusão de bancos numa eventual volta de silo para pool).
- Índices compostos começando por `tenant_id`, ex.: `(tenant_id, proposta_status)`. Sem isso, o filtro do RLS não usa índice e a leitura degrada com o crescimento.
- Sem `ON DELETE CASCADE` cruzando tenants.

6.2 Tabelas de plataforma

Tabela	Papel
<code>tenants</code>	Registry (seção 4.2). Sem RLS por tenant; acesso só pelo role administrativo
<code>usuarios</code>	Usuário, com <code>tenant_id</code> obrigatório
<code>usuario_sesoes</code>	Sessões. Nunca exposta por roteador genérico (lição do WebCRM)
<code>papeis / permissoes</code>	Autorização dentro do tenant
<code>auditoria</code>	Trilha append-only (seção 11.1)
<code>arquivos</code>	Metadados dos objetos no R2 (seção 9.3)

6.3 Tabelas de domínio de crédito

Esboço a ser detalhado com as regras de negócio. A modelagem de plataforma não depende deste detalhamento.

- `propostas` - solicitação de crédito, valor, prazo, status, esteira
- `proponentes` - pessoa física ou jurídica; campos sensíveis cifrados (seção 11.2)
- `analises` - cada avaliação feita sobre uma proposta, com entrada, saída e política aplicada
- `decisoos` - resultado formal, imutável após gravação
- `contratos` - crédito concedido
- `parcelas` - cronograma de pagamento
- `movimentos` - pagamentos, baixas e ajustes

Regra de imutabilidade: `decisoos` e `auditoria` não aceitam `UPDATE` nem `DELETE`. Correção se dá por novo registro que referencia o anterior, nunca por sobrescrita. Isso é exigência prática de reconstituição de decisão de crédito.

7. Isolamento: RLS em detalhe

7.1 Mecanismo

O backend abre uma transação e define o tenant do contexto antes de qualquer query:

```
BEGIN;  
SET LOCAL app.tenant_id = '<uuid do tenant da sessão>';  
-- queries da requisição  
COMMIT;
```

`SET LOCAL` é escopado à transação, o que é compatível com o pooler em modo transaction. A política de cada tabela compara o `tenant_id` da linha com esse valor:

```

ALTER TABLE propostas ENABLE ROW LEVEL SECURITY;
ALTER TABLE propostas FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON propostas
  USING      (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);

```

- `USING` filtra leitura, `UPDATE` e `DELETE`.
- `WITH CHECK` impede gravar linha de outro tenant.
- `FORCE ROW LEVEL SECURITY` aplica a política inclusive ao dono da tabela.

7.2 Papéis do banco

Role	Uso	RLS
<code>credx7i_owner</code>	Migrations, DDL. Nunca usado pela aplicação	N/A
<code>credx7i_app</code>	Conexão da aplicação. Sem <code>BYPASSRLS</code> , sem <code>SUPERUSER</code>	Sujeito
<code>credx7i_admin</code>	Rotinas de plataforma (provisionar tenant, relatório consolidado). Uso restrito e auditado	Isento onde necessário

7.3 Modos de falha que o RLS elimina

- Query nova esquecendo o `WHERE tenant_id = ...`.
- Tabela nova fora do mapa de permissão da aplicação (o achado real do WebCRM).
- SQL injection que consiga alterar o `WHERE`: a política é aplicada depois, pelo planner, e não é removível por manipulação de string.
- Bug de cache ou de reuso de conexão que misture contexto entre requisições, desde que o `SET LOCAL` esteja dentro da transação.

7.4 O que o RLS não resolve, e precisa de disciplina

- `current_setting('app.tenant_id', true)` retorna `NULL` se não definido, e a comparação com `NULL` resulta em zero linhas. É o comportamento seguro desejado (falha fechada), mas produz "sumiu tudo" difícil de diagnosticar. Deve haver um guard explícito no middleware que rejeita a requisição se o contexto não foi definido, com erro claro.
- Funções `SECURITY DEFINER` foram RLS por definição. Só usar com revisão explícita.
- Views não herdam RLS das tabelas base automaticamente em toda configuração. Views devem ser criadas com `security_invoker = true` (Postgres 15+).
- Migrations rodam como `credx7i_owner` e não são filtradas. Script de correção de dados precisa de cuidado manual.

7.5 Teste de isolamento obrigatório

Uma suíte automatizada, rodada em CI e antes de todo deploy de produção:

1. Criar dois tenants com dado conhecido.

2. Para cada tabela de negócio, autenticado como tenant A, tentar `SELECT` , `UPDATE` , `DELETE` e `INSERT` visando dado do tenant B. Esperado: zero linhas ou erro de policy.
3. Falha em qualquer tabela bloqueia o deploy.
4. Um teste que enumera as tabelas do `information_schema` e falha se alguma tabela de negócio estiver sem RLS habilitado. Isso cobre a tabela nova esquecida.

O item 4 é o que transforma "lembrar de aplicar RLS" em garantia mecânica.

8. Autenticação e autorização

8.1 Autenticação

Mantém o padrão do WebCRM, com os ajustes já aprendidos lá:

- E-mail e senha, hash com `bcrypt` ou `argon2` .
- Busca de e-mail case-insensitive, com `UNIQUE` também case-insensitive **no banco** (índice em `lower(email)`), não apenas no código. No WebCRM essa proteção mora no código e um `INSERT` direto fura.
- Sessão em tabela, com expiração. Token opaco, não JWT.
- Rate limiting por IP nas rotas de login e recuperação de senha. Atenção: em serverless na Vercel, contador em memória não é confiável entre instâncias e `req.ip` é o do proxy sem `trust proxy` configurado. Usar armazenamento compartilhado (tabela ou Upstash Redis free tier) e ler `x-forwarded-for` .
- Troca de senha revoga demais sessões.
- Política de senha a definir com o cliente. Mínimo de 8 caracteres foi o adotado no WebCRM e ficou como pendência lá.

8.2 Escopo de tenant na sessão

A sessão carrega `tenant_id` . Como cada usuário pertence a um único tenant, não há troca de contexto, não há seletor de tenant e não há caso de "usuário em dois clientes". Isso simplifica materialmente o modelo e deve ser mantido como restrição enquanto possível.

Validação de coerência: se o subdomínio acessado não corresponder ao `tenant_id` da sessão, a requisição é rejeitada e o evento é registrado na auditoria. É sinal de tentativa de acesso cruzado.

8.3 Autorização dentro do tenant

Papéis por tenant (`analista` , `gestor` , `admin_cliente` , por exemplo), com permissões por recurso e operação. Diferente do WebCRM, a checagem de permissão **não é a fronteira de segurança entre clientes** - essa é o RLS. Aqui ela é apenas controle de função dentro da empresa cliente, o que reduz o impacto de uma falha nessa camada.

8.4 Alternativa avaliada: Supabase Auth

Supabase Auth com `tenant_id` como claim no JWT e políticas usando `auth.jwt() ->> 'tenant_id'` é o caminho mais idiomático e elimina código de sessão. Não foi escolhido porque exigiria reescrever a

autenticação já dominada, acopla o projeto mais fortemente ao Supabase (dificultando o silo em outro provedor) e não traz ganho de custo. **Fica registrado como opção caso a auth própria mostre atrito.**

9. Upload e armazenamento de arquivos

9.1 Por que Cloudflare R2

Critério	Supabase Storage	Cloudflare R2
Incluído no plano	100 GB no Pro	10 GB grátis
Excedente	~US\$ 0,021/GB/mês	~US\$ 0,015/GB/mês
Egress	Cobrado acima da cota	Zero
Acoplamento	Amarrado ao projeto Supabase	Independente; sobrevive à troca de banco

No ano 3 (~90 GB), o R2 custa cerca de US\$ 1,20/mês. O egress zero é o fator decisivo: documentos de crédito são baixados repetidamente por analistas e auditores, e é aí que storage com egress cobrado surpreende na fatura. A independência em relação ao Supabase também é o que permite um tenant migrar para silo sem mover arquivo de provedor.

9.2 Fluxo de upload

Upload **direto do browser para o R2**, via URL pré-assinada. Isso não é otimização, é requisito: função serverless na Vercel tem limite de corpo de requisição em torno de 4,5 MB, e documento escaneado de crédito passa disso com facilidade.

```
1. Browser -> Backend: "quero enviar contrato.pdf, 8 MB, proposta X"
2. Backend: valida sessão, permissão, tipo MIME, tamanho
              gera chave <tenant_id>/<ano>/<proposta_id>/<uuid>.pdf
              grava linha em `arquivos` com status 'pendente'
              emite URL PUT pré-assinada, TTL 5 min
3. Browser -> R2:      PUT direto
4. Browser -> Backend: "concluído"
5. Backend: confirma existência e tamanho no R2, marca 'ativo'
```

Download segue o inverso: URL GET pré-assinada com TTL curto, emitida somente após checagem de sessão, tenant e permissão.

9.3 Regras

- **Bucket privado.** Nenhum objeto público. Nenhuma chave de acesso R2 chega ao browser, apenas URLs pré-assinadas.
- **Prefixo obrigatório** `<tenant_id>/` em toda chave. A chave é sempre construída no servidor a partir do `tenant_id` da sessão, nunca a partir de caminho enviado pelo cliente. Isso elimina path traversal entre tenants.

- **Tabela `arquivos`** com `tenant_id`, protegida por RLS. Ela é a fonte de verdade de autorização; o R2 é apenas o depósito de bytes. Um objeto sem linha correspondente é órfão e deve ser varrido por rotina periódica.
- **Nome original preservado como metadado**, nunca como caminho. O caminho é sempre UUID gerado no servidor.
- **Allowlist de tipo MIME e tamanho máximo** definidos por tipo de documento.
- **Antivírus**: avaliar na fase 4. Documento enviado por terceiro e baixado por analista é vetor real.

Nota do WebCRM: houve caso de registros de anexo apontando para caminhos que nunca foram do sistema, causando "anexo perdido". A tabela `arquivos` com confirmação de gravação (passo 5 acima) existe para evitar essa divergência entre metadado e objeto.

10. Ambientes, migrations e deploy

10.1 Ambientes

Ambiente	Banco	Vercel
Local	Postgres em Docker	<code>npm run dev</code> , portas fixas
Preview	Projeto Supabase separado, dados sintéticos	Deploy automático por branch
Produção	Projeto Supabase de produção	Branch <code>main</code>

Produção nunca compartilha banco com preview. No WebCRM, preview e produção apontando para o mesmo lugar já causou confusão de estado.

10.2 Migrations

Ponto de melhoria explícito sobre o WebCRM, onde alterações de schema são scripts `.sql` rodados manualmente no SQL Editor do Supabase.

Para um produto multitenant isso não escala: quando existirem silos, cada migration precisa rodar em N bancos, na ordem certa, com registro de qual já rodou.

Adotar ferramenta de migration versionada (`node-pg-migrate`, `dbmate` ou `drizzle-kit`) desde a fase 1, com:

- Migrations numeradas e versionadas no repositório.
- Tabela de controle do que já foi aplicado, por banco.
- Comando único que aplica migrations pendentes em todos os bancos do registry.
- Migrations idempotentes e reversíveis sempre que possível.

Observação prática herdada: o SQL Editor do Supabase exibe apenas o resultado da última instrução de um lote. Scripts de verificação devem terminar com o `SELECT` que importa.

10.3 Provisionamento de tenant

Rotina única e roteirizada, não sequência manual:

1. Inserir em `tenants` com `modo = 'pool'`.
2. Criar o subdomínio (wildcard já cobre; sem passo manual na Vercel).
3. Criar o prefixo no R2 (implícito, R2 não exige criação de "pasta").
4. Criar o primeiro usuário `admin_cliente` e enviar convite.
5. Aplicar dados iniciais (papéis padrão, parametrização de crédito).

Meta: cliente novo em minutos, sem alteração de código nem de infraestrutura.

11. Segurança e conformidade

11.1 Trilha de auditoria

Tabela `auditoria`, append-only, registrando no mínimo:

`tenant_id`, `usuario_id`, `ocorrido_em`, `acao`, `entidade`, `entidade_id`, `valor_anterior` (jsonb), `valor_novo` (jsonb), `ip`, `user_agent`.

- Sem `UPDATE` e sem `DELETE` concedidos ao role da aplicação.
- Gravada por trigger nas tabelas sensíveis, não por chamada explícita no código (chamada explícita é esquecida).
- Toda decisão de crédito deve ser reconstituível: quem decidiu, quando, com quais dados de entrada e sob qual versão de política. Sem isso não há defesa em contestação.
- Registrar também eventos de segurança: login, falha de login, troca de senha, emissão de URL pré-assinada, tentativa de acesso cruzado entre tenants.

11.2 Dados pessoais e LGPD

- **Criptografia em coluna** para CPF/CNPJ de proponente, renda e demais campos sensíveis. Avaliar `pgsodium` /Vault do Supabase ou cifragem na aplicação com chave por tenant. O objetivo é que um dump vazado não exponha o dado em claro.
- **Índice de busca:** campo cifrado não é pesquisável. Se houver busca por CPF, manter coluna adicional com hash determinístico (HMAC com chave do sistema) para igualdade exata, e cifrar o valor real.
- **Minimização:** não coletar nem reter o que a decisão de crédito não exige.
- **Retenção e expurgo:** política de retenção por tipo de dado, com rotina de expurgo automatizada. Necessário também para atender pedido de eliminação.
- **Portabilidade e encerramento:** rotina de exportação completa dos dados de um tenant. É requisito de LGPD e, convenientemente, é o mesmo mecanismo usado na promoção para silo e na saída de cliente.
- **Encarregado (DPO) e base legal:** definir com o jurídico. Fora do escopo técnico, mas bloqueia entrada em produção com dado real.

11.3 Backup e recuperação

- **PITR (point-in-time recovery)** habilitado. Está disponível apenas em plano pago, o que é mais um motivo para não operar produção em free tier.

- Teste de restauração executado ao menos uma vez antes do go-live, e depois periodicamente. Backup nunca testado não é backup.
- R2 com versionamento de objeto habilitado, protegendo contra sobrescrita e exclusão acidental.

11.4 Superfície de API

- Não replicar o roteador REST genérico sobre tabelas do WebCRM. Endpoints explícitos por caso de uso reduzem a superfície e evitam a classe de falha do `MENU_BY_RESOURCE`. Se o roteador genérico for reaproveitado por produtividade, o RLS passa a ser a única linha de defesa entre clientes, e a suíte da seção 7.5 vira crítica.
- Headers de segurança (HSTS, CSP, `X-Content-Type-Options`, `Referrer-Policy`) definidos em código, não apenas no painel da Vercel. No WebCRM essa decisão já foi tomada nesse sentido e provou ser a certa.
- CORS restrito à origem do frontend, com wildcard de subdomínio controlado.
- Validação de entrada com schema (`zod`) em toda rota.

12. Custos

12.1 Produção, 10 clientes

Item	Plano	Mensal (USD)
Supabase	Pro (8 GB banco, PITR, sem pausa)	25
Vercel	Pro (uso comercial)	20
Cloudflare R2	90 GB no ano 3, 10 GB grátis	~1
Domínio	rateado	~2
Total		~48

Cerca de **US\$ 4,80 por cliente por mês**. Com 10 clientes pagantes, o custo de infraestrutura é irrelevante frente à receita.

12.2 Fase de piloto

Free tier viável para desenvolvimento e demonstração: Vercel Hobby, Supabase Free, R2 Free. **Ressalvas:** Supabase Free pausa o projeto após dias sem atividade e não tem PITR; Vercel Hobby não permite uso comercial. Free tier serve para construir e demonstrar, não para atender cliente real.

12.3 Custo de um silo

Cada tenant em banco dedicado adiciona algo na ordem de **US\$ 10 a 25/mês**, dependendo do provedor e do tamanho de compute (Neon com scale-to-zero tende a ficar na ponta baixa; projeto Supabase adicional, na ponta alta). Valores a confirmar na contratação.

Recomendação comercial: isolamento dedicado deve ser item precificado no contrato, não cortesia. Além do custo direto, ele adiciona trabalho recorrente de operação.

12.4 Gatilhos de revisão de custo

- Banco passando de 8 GB: avaliar disco adicional ou arquivamento de histórico frio.
- Storage passando de 500 GB: renegociar ou revisar política de retenção.
- Mais de 3 silos: o custo operacional passa a justificar automação de fleet de bancos.

13. Observabilidade

- **Logs estruturados** em JSON com `request_id` e `tenant_id` em toda linha. Sem `tenant_id` no log, diagnosticar incidente em multitenant é inviável.
- **Nunca logar** dado pessoal, token de sessão, URL pré-assinada completa ou conteúdo de campo cifrado.
- **Métricas mínimas por tenant:** requisições, latência p95, erros 5xx, tamanho do banco, volume no R2. Servem para operação e para eventual cobrança por uso.
- **Alertas:** taxa de erro, falha de login em rajada, tentativa de acesso cruzado entre tenants (deve ser sempre zero; qualquer ocorrência é incidente).
- **Cold start:** já foi problema no WebCRM na Vercel. Monitorar e, se necessário, manter rota de aquecimento.

14. Riscos

#	Risco	Impacto	Mitigação
1	Tabela nova sem RLS habilitado	Vazamento entre clientes	Teste automatizado que varre <code>information_schema</code> e bloqueia deploy (7.5)
2	<code>SET LOCAL</code> fora da transação com pooler em modo transaction	Contexto vazando entre requisições	Toda query dentro de transação; middleware único, sem acesso direto ao pool
3	Reaproveitar o roteador REST genérico do WebCRM	Superfície ampla, repetindo falha conhecida	Endpoints explícitos; se reaproveitar, RLS vira defesa única e a suíte 7.5 é obrigatória
4	Cliente exigindo silo antes da fase 5	Retrabalho ou prazo comercial perdido	<code>tenant_id</code> em toda tabela desde a fase 1; registry desde a fase 2
5	Divergência de schema entre pool e silos	Bug intermitente por tenant	Migration versionada com controle por banco desde a fase 1 (10.2)
6	Chave de criptografia perdida	Dado sensível irrecuperável	Custódia da chave documentada, com backup fora da mesma infraestrutura
7	Free tier em produção	Projeto pausado, cliente sem serviço	Migrar para plano pago antes do primeiro cliente real
8	Autenticação própria com falha sutil	Comprometimento de conta	Revisão de segurança dedicada antes do go-live; considerar Supabase Auth (8.4)
9	Rate limit em memória em serverless	Proteção de login ineficaz	Contador em armazenamento compartilhado desde a fase 2

15. Roteiro de fases

Fase	Escopo	Entrega verificável
0. Fundação	Repositório, dois projetos Vercel, projeto Supabase, migration tool, <code>tenant_id</code> como convenção, CI	Deploy de "hello world" autenticado em preview
1. Isolamento	Tabela <code>tenants</code> , RLS em todas as tabelas, roles do banco, middleware de contexto, suíte de teste de isolamento	Dois tenants de teste sem enxergar dado um do outro, provado por teste automatizado
2. Identidade	Auth própria, sessões, rate limiting compartilhado, papéis e permissões, convite de usuário	Login por subdomínio, com escopo de tenant correto
3. Domínio de crédito	Modelo de propostas, análises, decisões, contratos, parcelas. Trilha de auditoria por trigger	Fluxo de proposta ponta a ponta
4. Arquivos	Integração R2, URLs pré-assinadas, tabela <code>arquivos</code> , validação de tipo e tamanho, rotina de órfãos	Upload de 10 MB direto do browser, download autorizado
5. Multi-banco	Registry com <code>modo = 'silo'</code> , resolução de conexão por request, migration em N bancos, roteiro de promoção	Um tenant de teste rodando em banco separado, mesmo código
6. Conformidade	Criptografia em coluna, retenção e expurgo, exportação de tenant, PITR, revisão de segurança	Checklist de segurança aprovado, restauração testada
7. Operação	Logs com <code>tenant_id</code> , métricas, alertas, provisionamento roteirizado de cliente	Cliente novo provisionado em minutos

Fases 0 e 1 são pré-requisito de tudo. É tentador começar pelo domínio de crédito, que é o que o cliente enxerga, mas retrofitar isolamento em base já povoada é caro e arriscado.

A fase 5 pode ser adiada até existir demanda real de silo, **desde que** as fases 0 e 1 tenham mantido `tenant_id` em toda tabela. Essa é a apólice de seguro do plano.

16. Decisões em aberto

Itens que dependem de definição do usuário ou do negócio:

- Domínio definitivo** do produto (assumido `credx7i.com.br` neste documento).
- Auth própria ou Supabase Auth** (seção 8.4). Recomendação: própria, mas vale reavaliar se a fase 2 mostrar atrito.
- Política de senha** (o WebCRM ficou em mínimo de 8 caracteres, sem exigência de complexidade, e isso permanece como pendência lá).
- Modelo de papéis** dentro do tenant: quais funções existem no processo de crédito do cliente.
- Tipos de documento** aceitos, tamanho máximo e política de retenção por tipo.

6. **Requisito regulatório específico** além de LGPD: há exigência de BACEN ou de órgão setorial aplicável ao produto de crédito em questão? Isso pode alterar retenção, auditoria e residência de dados.
 7. **Necessidade de MFA** para perfis com poder de decisão de crédito.
 8. **Antivírus em upload** (seção 9.3): necessário na fase 4 ou aceitável adiar?
 9. **Motor de decisão de crédito**: regras internas, integração com bureau, ou ambos. Impacta fortemente o modelo de `analises` e `decisoos`.
-

17. Referências internas

- `C:\Claude\WebCRM\STATUS.md` - histórico completo da arquitetura de referência, incluindo as auditorias de segurança e os achados citados neste documento.
- `C:\Claude\WebCRM\DESIGN_SYSTEM.md` - convenções de UI reutilizáveis.
- `C:\Claude\Check-list Vulnerabilidades WEB.md` - checklist de segurança já usado.
- `C:\Claude\WebCRM\schema.pg.sql` - referência de estilo de schema Postgres.